ASYNC + TS little CHALLENGE

“Queue” coding challenge

Author	
Created	@June 20, 2022 11:34 AM
Type	
Archived	☐

Problem

Sometimes we face the situation when we have to process multiple asynchronous tasks of the same type. For example, we want to (frequently) send a query to database with some `id`s. To simplify the example, let’s imagine that our table has the `value` column with the same value as `id` column, but with different type — string. This is how it could look like:

id	value
1	"1"
2	"2"
…	…
999	"999"

It’s not necessary to deploy real database for this challenge, so, we can simulate it with something like:

```
// There's is chance some requests are failing, so `null` is returned (sometimes).

function get(ids: number[]): Promise<(string | null)[]> {
  return new Promise((resolve) => {
    setTimeout(() => {
      const results = ids.map(id => Math.random() > 0.5 ? null : String(id))
      resolve(results)
    }, 1000)
  })
}
```

The problem is that we receive those tasks (in our example — ids) from users really frequently. So, we need to limit the rate of requesting and implement some kind of queue.

Requirement #1: rate limiting

It should be possible to set the amount of parallel requests. If there are more tasks, than parallels — the rest of the tasks should be enqueued and processed after some earlier request is completed. Also, it should be possible to enqueue each task separately and get Promise instance:

```
const task = 3
const response = await queue.exec(task) // response === "3"
```

Requirement #2: grouped processing

Pretty common situation is when there is the possibility of grouping tasks into single processing. In our concrete example, instead of requesting 5 ids separately, we can group them into single request.

```
# So, instead of
SELECT value FROM some_values WHERE id=1
SELECT value FROM some_values WHERE id=2
SELECT value FROM some_values WHERE id=3
SELECT value FROM some_values WHERE id=4
SELECT value FROM some_values WHERE id=5
```

```
# we can just
SELECT value FROM some_values WHERE id=1 OR id=2 OR id=3 OR id=4 OR id=5
```

Although grouping can help to optimize processing, we want to limit the maximum amount of grouped tasks too. Let's say we don't want to send more ids, than 100.

Requirement #3: simple interface

Although we will have multiple internal optimizations and abstractions, we still want queue to have as simple interface as possible. So, there is an example of how it should be possible to instantiate queue and process task(s):

```
// Instantiate
const queue = new Queue({
  name: 'Queue name',
  parallels: 5, // amount of parallels
  drainLimit: 3, // amount of tasks to group (if possible) (null if unlimited)
  processor: someFunction // the main processing function, see below
})

// Function responsible to process/group tasks.
(tasks: T[]) => Promise<SafeResult<U>[]>

// ...where SafeResult is:
type SafeResult<T> = { error: Error, value: T }

// process task:
const result = await queue.exec("some task")
```

If you are familiar of how `Promise.allSettled` works, you can expect what `SafeResult` is for. Basically, it's simple wrapping object of the result of the single task which sometimes contain error (if there was an error) or value (if everything completed successfully). Later on, queue should somehow manage those grouped results (`SafeResult`s) and resolve/reject corresponding promises.

Example:

For the given example (*parallels: 5, drainLimit: 3*) it could work as following:

- Gradually, we got some tasks from users, so there are 4 tasks enqueued (for example: *5, 8, 19, 9*)
- One parallel is freed and became accessible.
- Because `drainLimit` is set to `3` we will get `3` tasks from `4` enqueued and send it into processor function.
- So, processor takes *5, 8 and 19*. Inside of the function it groups it into single database request:

```
SELECT value FROM some_values WHERE id=5 OR id=8 OR id=19
```

- Task *19* fails, the rest is okay.
- This is what processor should return:

```
Promise<[
  { error: null, value: '5' },
  { error: null, value: '8' },
  { error: Error, value: undefined }
]>
```

- Based on those returned values queue should decide which promises to resolve/reject with given values.

Interface:

Queue should implement the following interface:

```
interface Queue<T, U> {
  exec(task: T): Promise<U>
  get status(): { name: string; available: number; working: number; enqueued: number }
}
```

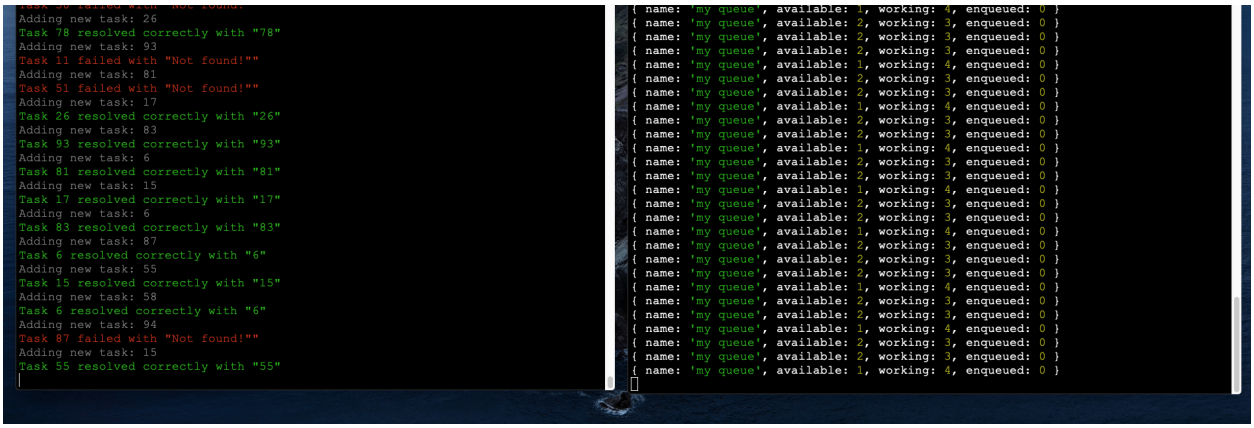
`status` getter should output something like (useful for debugging):

```
console.log(queue.status)

// {
//   name: 'Queue name',
//   available: 0, // available parallels
//   working: 5, // busy parallels
//   enqueued: 39 // awaiting tasks
// }
```

Useful types + db simulator:

```
type SafeResult<T> = { error: Error, value: T }

type QueueOptions<T, U> = {
  name: string
  parallels: number
  drainLimit: number
  processor: (tasks: T[]) => Promise<SafeResult<U>[]>
}

interface IQueue<T, U> {
  exec(task: T): Promise<U>
  readonly status: {
    name: string,
    available: number,
    working: number,
    enqueued: number
  }
}

class Queue<T, U> implements IQueue<T, U> {
  // ...
}

function get(ids: number[]): Promise<(string | null)[]> {
  return new Promise((resolve) => {
    setTimeout(() => {
      const results = ids.map(id => Math.random() > 0.5 ? null : String(id))
      resolve(results)
    }, 1000)
  })
}
```

Simple testing script:




```
// initialization
const queue = new Queue({
  name: 'some queue',
  parallels: 5,
  drainLimit: 3,
  processor: someFunction
})

// Each ~100ms enqueue new task and print when ready.
setInterval(async () => {
  const id = Math.floor(Math.random() * 100)
  console.log('New task: ', id)

  try {
    const value = await queue.exec(id)
    console.log('Task', id, 'resolved with', value)
  } catch(err) {
    console.log('Task', id, 'rejected with', err.message)
  }
}, 100)

// Each ~1000ms print queue status to stderr
setInterval(() => {
  console.error(q.status)
}, 1000)
```

We use `stderr` just to pipe it to the separate terminal tab. To pipe it, just do in separate tab:

```
// tab 2:
$ tty // /dev/ttys001

// tab 1:
ts-node index.ts 2>/dev/ttys001
```